

# Spoken Language Identification from Audio Using Deep Learning

Biniyam Zergaw

*Department of Computer and Information Science*

*Fordham University*

New York, NY, USA

bz24@fordham.edu

**Abstract**—This project studies spoken language identification: given a short speech recording, the goal is to predict the language being spoken. The final system supports nine languages: Amharic, Arabic, Chinese, English, French, Hindi, Italian, Russian, and Spanish. Audio recordings are converted into 128-bin log-Mel spectrograms and classified using neural network models implemented in PyTorch. The main model is a residual convolutional neural network (CNN), and a bidirectional long short-term memory (LSTM) model is used as an alternative sequence-based baseline. The training data combines Mozilla Common Voice style recordings with Google Few-shot Learning Evaluation of Universal Representations of Speech (FLEURS) recordings, using up to 10,000 primary clips and 5,000 FLEURS clips per language. On the held-out test set, the CNN achieved 89.3% accuracy, outperforming the LSTM baseline, which achieved 80.5% accuracy. Additional sample prediction tests showed that the CNN generalized better than the LSTM, although real microphone recordings remained more challenging because of domain mismatch, microphone noise, and recording conditions.

**Index Terms**—spoken language identification, audio classification, convolutional neural networks, LSTM, log-Mel spectrograms, PyTorch

## I. INTRODUCTION

Spoken language identification is the task of determining which language is being spoken from an audio recording. This problem is useful for multilingual speech interfaces, call routing, transcription systems, and mobile applications that must first identify a language before applying language-specific speech recognition. Unlike text language identification, the input is a waveform rather than written words, so the model must learn acoustic patterns related to phonetics, rhythm, pronunciation, and recording conditions.

The objective of this project is to build and evaluate a neural network system that can classify short audio clips into one of several spoken languages. The project was designed around topics covered in the course, especially neural networks, convolutional networks, nonlinear optimization, model selection, and Principal Component Analysis (PCA) visualization. The final system focuses on nine languages while still keeping training time manageable.

The central research question is:

Can a neural network trained on log-Mel spectrograms identify the spoken language of a short recording across a multilingual dataset, and how does a CNN compare against an LSTM baseline?

## II. PROBLEM FORMULATION

Let  $x$  be an audio waveform and let  $y \in \{1, \dots, 9\}$  be the language label. The task is formulated as a supervised multi-class classification problem. Each waveform is resampled to 16 kHz, converted into a fixed-length five-second representation, transformed into a log-Mel spectrogram, and passed to a neural network classifier. The classifier outputs a probability distribution over the nine languages:

$$\hat{y} = \arg \max_k p(y = k | x). \quad (1)$$

The training objective is cross-entropy loss:

$$\mathcal{L} = - \sum_{k=1}^K y_k \log(\hat{p}_k), \quad (2)$$

where  $K = 9$ ,  $y_k$  is the one-hot encoded true label, and  $\hat{p}_k$  is the predicted probability for class  $k$ .

## III. DATASET

The dataset is organized by language under `data/raw/{language}/`. The project uses two main sources. The primary source is Mozilla Common Voice style audio [1]. The second source is Google FLEURS [2], which adds a different recording domain and helps test cross-dataset robustness. For Amharic, an auxiliary Amharic speech source is used to supplement the primary data because the Mozilla Amharic data was much smaller than the other language datasets.

The dataset was capped at the following maximum source counts:

- up to 10,000 primary Mozilla Common Voice clips per language.
- up to 5,000 FLEURS clips per language.
- up to 15,000 total clips per language before filtering and splitting.

The dataset loader recursively scans each language folder and treats files inside a `fleurs` subfolder as FLEURS data. This allows the training split to include both primary and FLEURS clips while still keeping the sources identifiable.

After filtering and source availability differences, the final split sizes were:

TABLE I  
DATASET SPLIT SIZES

| Split      | Number of clips |
|------------|-----------------|
| Training   | 80,371          |
| Validation | 17,222          |
| Test       | 17,223          |

#### IV. FEATURE EXTRACTION AND PREPROCESSING

The system uses log-Mel spectrograms rather than raw waveforms as the main model input. Each audio file is loaded with `librosa` [4], converted to mono audio, resampled to 16 kHz, and represented as a five-second clip. A 128-bin Mel-spectrogram is computed with an FFT (Fast Fourier Transform) size of 1024 and hop length of 512. The Mel power spectrogram is then converted to decibel scale and normalized.

Log-Mel spectrograms were selected because they preserve local time-frequency structure. This makes them a natural input for convolutional networks, where filters can learn acoustic patterns such as harmonics, formants, and phonetic transitions. MFCC extraction was implemented as a secondary feature option, but the final CNN and LSTM experiments use log-Mel spectrograms.

To improve performance, the project includes several pre-processing and augmentation options:

- silence trimming using voice activity style thresholding;
- random five-second cropping during live augmentation runs;
- random gain, background noise, pitch shift, and time stretch;
- SpecAugment-style frequency and time masking.

In the final stable training pipeline, deterministic Mel-spectrograms are cached to reduce training time. SpecAugment masks are still applied during training. The heavier waveform augmentation path is available, but it is slower because it requires decoding and transforming audio live in every epoch.

#### V. MODEL ARCHITECTURES

##### A. Residual CNN

The primary model is a residual CNN trained on log-Mel spectrograms. The input shape is approximately  $(1, 128, T)$ , where 128 is the number of Mel bands and  $T$  is the number of time frames. The CNN contains five convolutional blocks:

- Block 1:  $1 \rightarrow 32$  channels
- Block 2:  $32 \rightarrow 64$  channels
- Block 3:  $64 \rightarrow 128$  channels
- Block 4:  $128 \rightarrow 256$  channels
- Block 5:  $256 \rightarrow 512$  channels

Each block uses two  $3 \times 3$  convolutions, batch normalization, ReLU activation, and a residual skip connection. Max pooling reduces the spatial resolution in earlier blocks, and adaptive average pooling produces a compact 512-dimensional representation. The classifier uses fully connected layers  $512 \rightarrow 256 \rightarrow 128 \rightarrow 9$  with dropout.

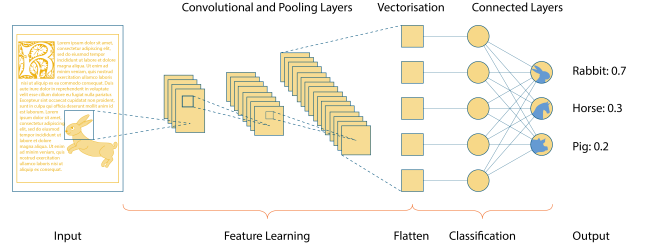


Fig. 1. General convolutional neural network structure [5].

TABLE II  
MAIN TRAINING CONFIGURATION

| Parameter               | Value              |
|-------------------------|--------------------|
| Sample rate             | 16 kHz             |
| Clip duration           | 5 seconds          |
| Mel bands               | 128                |
| Batch size              | 64                 |
| Learning rate           | $3 \times 10^{-4}$ |
| Weight decay            | $1 \times 10^{-4}$ |
| Maximum epochs          | 30                 |
| Early stopping patience | 8                  |
| Training workers        | 4                  |

This architecture is appropriate for spectrogram classification because convolutional filters can learn local frequency-time patterns and combine them hierarchically into language-level representations. Figure 1 illustrates the general structure of a convolutional neural network.

##### B. Bidirectional LSTM Baseline

The alternative model is a bidirectional LSTM. The spectrogram is interpreted here as a sequence over time: each time frame is represented by its 128 Mel coefficients. The LSTM has two bidirectional layers with hidden size 128, followed by a fully connected classifier.

The LSTM was included as a comparison because recurrent networks are designed for sequential data. However, it has fewer parameters than the CNN and trains more slowly because recurrent operations are less parallelizable than convolutions. In this project, the LSTM provided a useful baseline but did not match the CNN's accuracy.

#### VI. TRAINING SETUP

Both models were trained using PyTorch [3] with cross-entropy loss and the Adaptive Moment Estimation (Adam) optimizer. The main hyperparameters were:

Feature caching was used to make training practical. Pre-computing or lazily caching Mel-spectrogram tensors reduces repeated audio decoding and feature extraction.

#### VII. EVALUATION METHOD

The main evaluation uses the held-out test split, which contains 17,223 clips. Performance is measured with accuracy, precision, recall, and F1-score. A confusion matrix is used to inspect which languages are confused with each other. PCA is

TABLE III  
INITIAL SMALL-DATA TEST PERFORMANCE

| Language | Precision                     | Recall | F1   |
|----------|-------------------------------|--------|------|
| Italian  | 0.58                          | 0.84   | 0.69 |
| Russian  | 0.79                          | 0.48   | 0.59 |
| Amharic  | 0.92                          | 0.81   | 0.86 |
| Hindi    | 0.78                          | 0.80   | 0.79 |
| Overall  | approximately 70–72% accuracy |        |      |

applied to the model embeddings to visualize whether examples from the same language form clusters in representation space.

In addition to the held-out test set, generated sample predictions were used as a smaller practical test. The sample prediction script runs inference on audio files in `samples/predict_test/` and reports accuracy per-language and overall. Additionally, users can test microphone recordings qualitatively in a notebook. Predictions on real-time recordings were more difficult than predictions on the held-out test set because user-recorded speech introduces real-world variation and additional noise.

## VIII. RESULTS

### A. Initial Small-Data Sanity Test

Before training the final nine-language system, an initial small-data experiment was used as a sanity test for the architecture and training pipeline. This run used a smaller language set and fewer samples. The goal was not to produce the final model, but to answer a more basic question: could the neural network architecture learn language-discriminative information from spectrograms at all?

The initial model did learn the training distribution, but the results also showed signs of overfitting and weak generalization. On the initial held-out test set, the model reached roughly 70–72% accuracy across four languages. The per-class results are shown in Table III. Amharic was the strongest class, while Russian had low recall, meaning many Russian clips were classified as another language. Italian had high recall but low precision, which means the model often predicted Italian for clips that were not actually Italian.

The initial generated-sample test was weaker: only 24 out of 45 samples were classified correctly, or 53.3% accuracy. This was a useful diagnostic result. It showed that the model was capable of fitting the dataset split, but it had not learned features that generalized reliably to samples from different sources. In other words, the early model was not just underfitting; it was learning dataset-specific patterns. This motivated the later changes: adding more languages, expanding the number of clips, adding FLEURS data, improving preprocessing, and comparing the CNN against an LSTM baseline.

The initial confusion matrix and learning curves are shown in Figures 2 and 3.

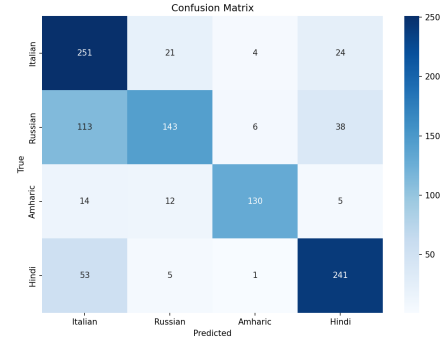


Fig. 2. Confusion matrix for the initial CNN model.

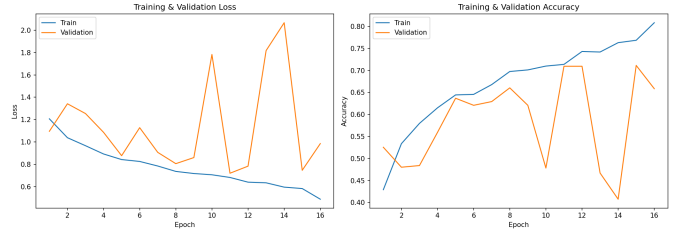


Fig. 3. Training and validation curves for the initial overfit CNN model.

### B. CNN Results

The residual CNN achieved 89.3% overall accuracy on the held-out test set. Table IV summarizes the per-language F1-scores. The CNN confusion matrix and learning curves are shown in Figures 4 and 5.

### C. LSTM Results

The LSTM baseline achieved 80.5% overall accuracy. It performed well on some languages, such as Amharic, but generally underperformed the CNN. This suggests that for this feature representation, local time-frequency convolutional patterns are more effective than treating the spectrogram only as a sequence of frames. The LSTM confusion matrix and learning curves are shown in Figures 6 and 7.

### D. Model Comparison

Table VI compares the two neural network approaches. The CNN outperformed the LSTM on both the held-out test set

TABLE IV  
CNN HELD-OUT TEST PERFORMANCE

| Language | Precision      | Recall | F1   |
|----------|----------------|--------|------|
| Amharic  | 0.98           | 0.98   | 0.98 |
| Arabic   | 0.91           | 0.88   | 0.89 |
| Chinese  | 0.91           | 0.94   | 0.92 |
| English  | 0.87           | 0.80   | 0.83 |
| French   | 0.86           | 0.90   | 0.88 |
| Hindi    | 0.91           | 0.92   | 0.92 |
| Italian  | 0.89           | 0.86   | 0.87 |
| Russian  | 0.86           | 0.90   | 0.88 |
| Spanish  | 0.85           | 0.85   | 0.85 |
| Overall  | 89.3% accuracy |        |      |

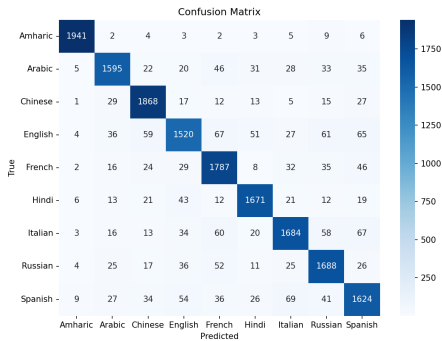


Fig. 4. Confusion matrix for CNN model.

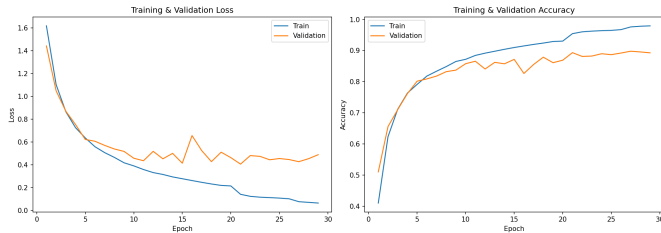


Fig. 5. Training and validation curves for CNN model.

TABLE V  
LSTM HELD-OUT TEST PERFORMANCE

| Language | Precision      | Recall | F1   |
|----------|----------------|--------|------|
| Amharic  | 0.97           | 0.96   | 0.97 |
| Arabic   | 0.76           | 0.79   | 0.77 |
| Chinese  | 0.80           | 0.88   | 0.84 |
| English  | 0.67           | 0.83   | 0.74 |
| French   | 0.89           | 0.73   | 0.80 |
| Hindi    | 0.84           | 0.83   | 0.84 |
| Italian  | 0.74           | 0.77   | 0.75 |
| Russian  | 0.82           | 0.75   | 0.78 |
| Spanish  | 0.79           | 0.71   | 0.75 |
| Overall  | 80.5% accuracy |        |      |

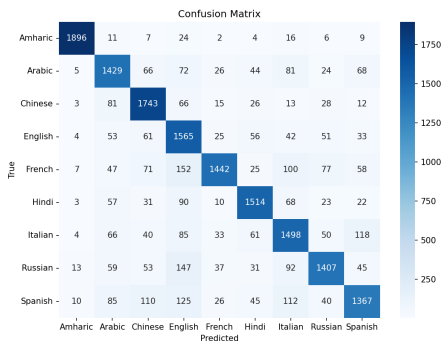


Fig. 6. Confusion matrix for the LSTM model.

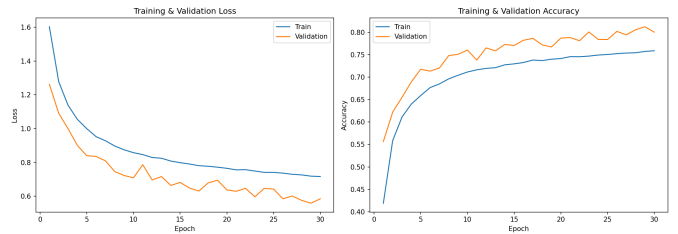


Fig. 7. Training and validation curves for LSTM model.

TABLE VI  
CNN AND LSTM COMPARISON

| Model              | Held-out accuracy | Recorded Sample accuracy |
|--------------------|-------------------|--------------------------|
| Residual CNN       | 89.3%             | 84.4%                    |
| Bidirectional LSTM | 80.5%             | 71.1%                    |

and the generated sample prediction set.

The CNN is consistently better with the current structure of the input. A spectrogram is image-like: neighboring time-frequency regions carry meaningful local patterns. CNNs are designed to detect these local patterns efficiently. The LSTM, by contrast, processes the spectrogram as a sequence and may not capture local frequency relationships as effectively.

## IX. INTERPRETATION AND DISCUSSION

The results show that the CNN is the stronger model for our architecture. It achieves high held-out accuracy across nine languages and maintains a clear advantage over the LSTM baseline. The best classes were Amharic, Chinese, and Hindi, while English, Spanish, Italian, French, Russian, and Arabic showed slightly higher confusion. Some of this confusion is expected because many of these languages share phonetic or prosodic similarities, and some recording domains may be more similar across languages than others.

The confusion matrices are useful because they show more than the overall accuracy. They reveal which mistakes are systematic. The model is least confused on languages with more distinctive acoustic or phonetic patterns in the dataset, such as Amharic and Chinese. It is more likely to confuse languages that are acoustically or linguistically closer. For example, the Romance languages in the dataset, French, Italian, and Spanish, share related phonetic and prosodic properties, so some confusion among them is expected. Figure 8 gives one linguistic view of relatedness among European languages, although spoken-language confusion also depends on acoustic and recording conditions. The learned signal can also be influenced by speaker, recording quality, or dataset-specific background characteristics rather than only language identity. English had lower recall than expected in the final CNN report, suggesting that some English clips were mapped into neighboring acoustic regions rather than forming a perfectly separated cluster.

This interpretation is consistent with the PCA embedding visualization. A good embedding should place clips from

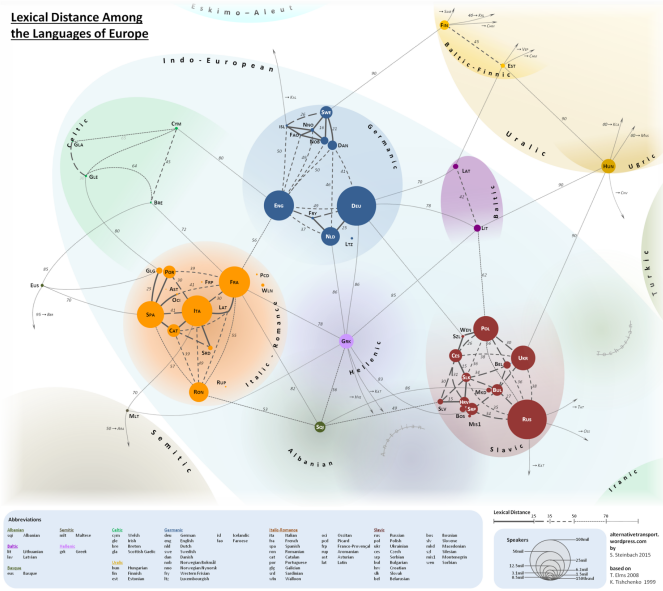


Fig. 8. Lexical distances between European languages [6].

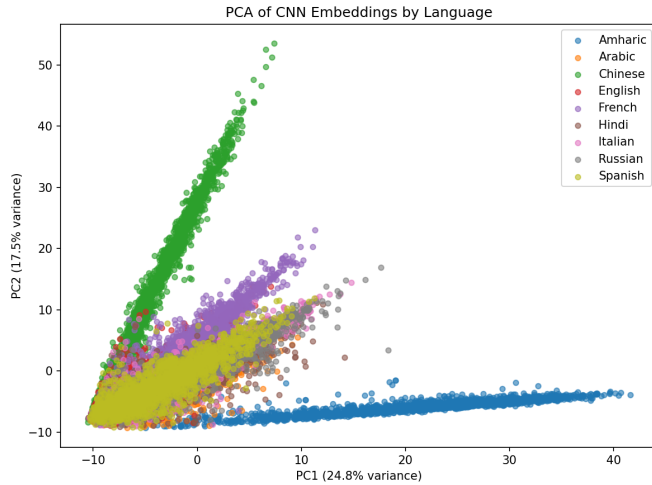


Fig. 9. PCA visualization of CNN embeddings for the nine-language classification system.

the same language close together, but overlapping clusters indicate classes that the model has difficulty separating. The confusion matrix and PCA plot in Figure 9 therefore provide complementary evidence: the model has learned meaningful language structure, but some language boundaries remain soft rather than perfectly separable.

Real microphone recordings were more difficult than the held-out test set. This is an important finding. The held-out test split is drawn from the same dataset distribution as training, while microphone recordings introduce new conditions: different microphones, background noise, silence at the beginning or end of speech, room echo, spontaneous pronunciation, and lower audio quality. This domain mismatch explains why a model can reach 89.3% accuracy on the test set but still make

mistakes on user-recorded audio.

The project also demonstrated important engineering trade-offs. Live waveform augmentation can improve robustness, but it greatly increases training cost. Cached deterministic Mel-spectrograms made training more stable, while SpecAugment still provided some training variation. This balance allowed both CNN and LSTM models to be trained and compared.

## X. LIMITATIONS

The main limitation in this domain is variation. Real recordings differ by environment, microphone, background noise, speaker identity, accent, speaking style, and silence before or after the target speech. The training data comes mainly from public speech datasets, which provide useful coverage but cannot fully represent all real-world recording conditions. A more robust system would require either more diverse target-domain recordings or stronger augmentation methods that simulate these variations during training.

Another limitation is that both neural network models were trained from scratch. Modern speech systems often use pretrained encoders such as wav2vec 2.0, Hidden-Unit BERT (HuBERT), WavLM, or Whisper, which have already learned robust speech representations from very large datasets. Fine-tuning one of these encoders would likely improve performance on noisy microphone recordings and out-of-domain clips.

The LSTM baseline was useful for comparison, but it does not represent the strongest possible sequence model. A larger recurrent model, an attention-based architecture, or a pretrained transformer encoder could learn richer temporal patterns than the simple LSTM baseline used here.

## XI. FUTURE WORK

Several improvements could make the system more robust:

- Increase the size and diversity of the training dataset.
- Apply stronger augmentation during training to simulate realistic recording variation.
- Use multi-window prediction by averaging predictions across several time windows from the same recording.
- Fine-tune a pretrained speech encoder such as wav2vec 2.0, HuBERT, WavLM, or Whisper.

The most promising gains would likely come from using larger and more diverse training data together with transfer learning from a pretrained speech model. This would likely improve noisy microphone performance by a significant amount.

## XII. CONCLUSION

This project formulated spoken language identification as a supervised multi-class audio classification problem and implemented a complete neural network pipeline in PyTorch. The system supports nine languages, combines Mozilla-style and FLEURS data, extracts log-Mel spectrograms, trains CNN and LSTM models, and evaluates performance with classification reports, confusion matrices, PCA visualizations, and sample predictions.

The residual CNN achieved the best performance, with 89.3% held-out test accuracy and 84.4% generated sample accuracy. The LSTM baseline achieved 80.5% held-out test accuracy and 71.1% generated sample accuracy. These results support the conclusion that CNNs are better suited than the tested LSTM for log-Mel spectrogram classification in this project. At the same time, the gap between dataset performance and microphone performance shows that robust real-world language identification requires either target-domain recordings, stronger augmentation, or transfer learning from pretrained speech models.

#### REFERENCES

- [1] Mozilla Common Voice, “Common Voice Dataset,” 2024. [Online]. Available: <https://commonvoice.mozilla.org/>
- [2] A. Conneau et al., “FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech,” 2022.
- [3] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems*, 2019.
- [4] B. McFee et al., “librosa: Audio and Music Signal Analysis in Python,” in *Proceedings of the 14th Python in Science Conference*, 2015.
- [5] Wikimedia Commons, “Convolutional Neural Network,” [Online]. Available: <https://commons.wikimedia.org/>
- [6] S. F. Steinbach, “Lexical Distances Between Europe’s Languages,” [Online]. Available: <https://alternativetransport.wordpress.com/>